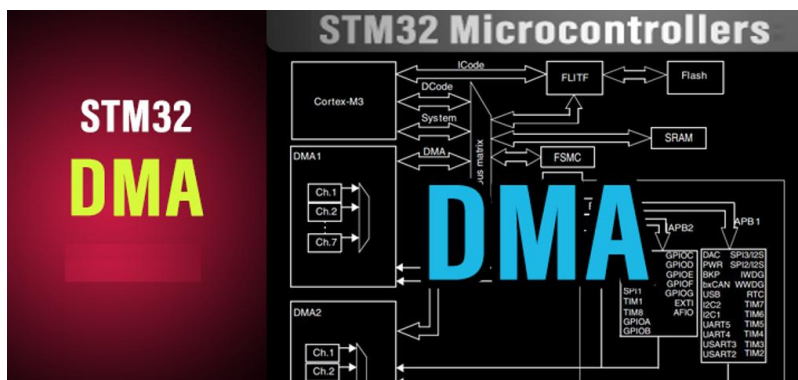


1. DIRECT MEMORY ACCESS (DMA):

1.1. Giới thiệu tổng quan về DMA (Direct Memory Access)

1.1.1. Khái niệm:

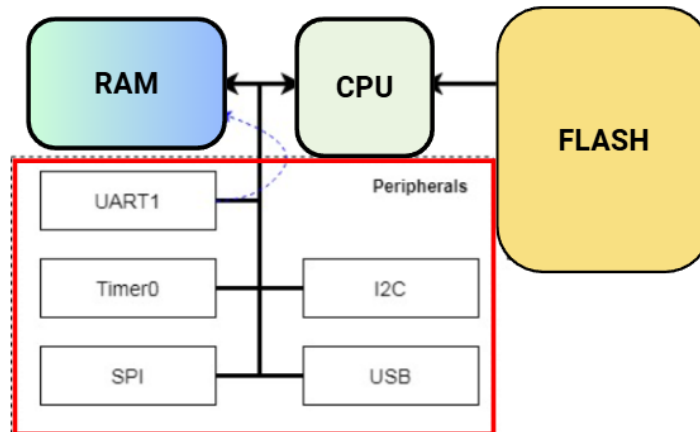


- **Direct Memory Access (DMA)** là một tính năng phần cứng chuyên dụng cho phép các ngoại vi (Peripherals) truy cập trực tiếp vào bộ nhớ (SRAM) để đọc/ghi dữ liệu mà không cần sự can thiệp của CPU cho từng byte dữ liệu.
- Trong các hệ thống nhúng thời gian thực (Real-time Embedded Systems), DMA đóng vai trò tối quan trọng:
 - **Giảm tải CPU:** CPU được giải phóng khỏi các tác vụ di chuyển dữ liệu nhằm chần để tập trung vào xử lý thuật toán phức tạp hoặc đi vào chế độ ngủ (Sleep Mode) để tiết kiệm năng lượng.
 - **Tăng thông lượng (Throughput):** Dữ liệu được chuyển đi với tốc độ tối đa của Bus hệ thống.

1.1.2. So sánh với các phương pháp khác:

Phương pháp	Cơ chế hoạt động	Ưu điểm	Nhược điểm
Polling (Vòng lặp)	CPU liên tục kiểm tra cờ trạng thái và thực hiện lệnh MOV dữ liệu.	Đơn giản, dễ lập trình.	Chiếm dụng toàn bộ thời gian CPU, lãng phí tài nguyên.
Interrupt (Ngắt)	Ngoại vi báo ngắt, CPU tạm dừng việc chính để xử lý ISR (Interrupt Service Routine).	CPU rảnh khi không có dữ liệu.	Overhead lớn khi chuyển ngữ cảnh (Context Switching) nếu tần suất ngắt quá cao.
DMA	Bộ điều khiển DMA tự động chuyển dữ liệu. CPU chỉ can thiệp khi hoàn tất khối dữ liệu lớn.	Hiệu suất cao nhất, CPU rảnh rỗi.	Cấu hình phức tạp hơn.

- Sơ đồ minh họa kiến trúc **KHI KHÔNG CÓ DMA (No-DMA Architecture)**:



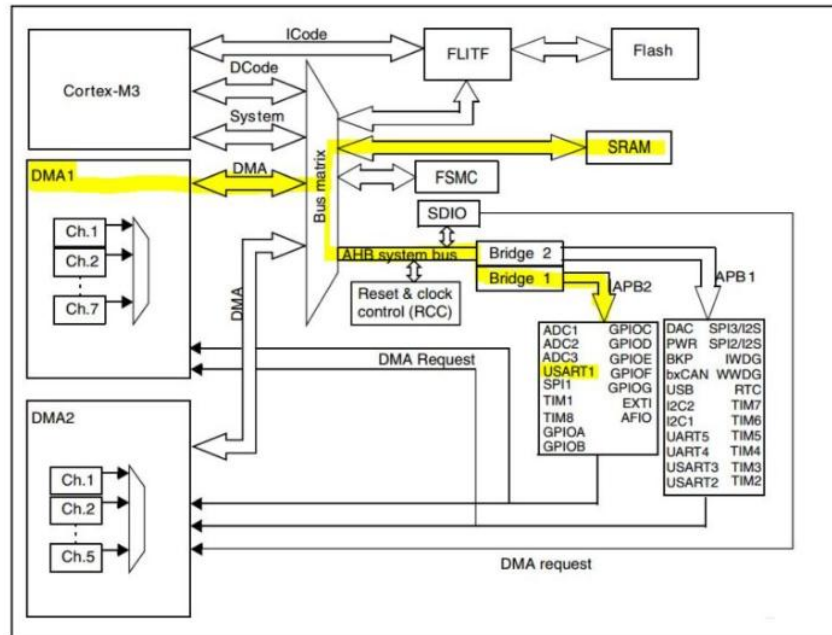
- Trong đó, chức năng từng khối như sau:
 - **FLASH:** Đây là nơi chứa đoạn code bạn nạp vào (Code C/C++ sau khi biên dịch).
Vai trò: CPU liên tục phải "nhìn" vào đây để biết dòng lệnh tiếp theo phải làm gì.
 - **RAM:** Đây là nơi chứa các biến số (variables), mảng (arrays), bộ đệm (buffers).
Vai trò: Dữ liệu từ cảm biến hay UART sau cùng phải nằm ở đây để bạn xử lý.
 - **CPU (Central Processing Unit):** Nhân ARM Cortex-M3.
Vai trò: "Ông chủ" kiêm "cửu vạn". Vừa phải đọc lệnh từ FLASH để tính toán, vừa phải chạy đi bung bê dữ liệu.
 - **Peripherals (UART1, SPI, I2C...):** Các ngoại vi giao tiếp.
Trong ảnh, tác giả vẽ **UART1** có một đường đứt nét màu xanh đi ra. Đây là ví dụ về việc nhận dữ liệu từ máy tính hoặc module khác.
- Đường đi dữ liệu được tiến hành như sau:
 - **Bước 1 - Xuất phát (UART1):** Một byte dữ liệu (ví dụ chữ 'A') bay từ máy tính vào chân RX của STM32, chui vào thanh ghi dữ liệu của UART1 (USART1->DR).
 - **Bước 2 - Đi qua CPU:** Đây là điểm mấu chốt! Mũi tên hướng về phía thanh trục (Bus) và đi lên CPU.
 - Lúc này, CPU phải ngừng việc đang làm (ví dụ đang tính toán PID cho động cơ).
 - CPU thực hiện lệnh LDR (Load Register) để đọc chữ 'A' từ UART vào thanh ghi nội bộ của nó (R0, R1...).
 - **Kết thúc (RAM):** Từ CPU, mũi tên hướng sang RAM.

- CPU thực hiện lệnh STR (Store Register) để ghi chữ 'A' đó vào biến trong RAM.

⇒ Kết luận: Dữ liệu muốn đi từ UART sang RAM bắt buộc phải "quá cảnh" qua CPU. CPU trở thành cái "trạm trung chuyển" bất đắc dĩ.

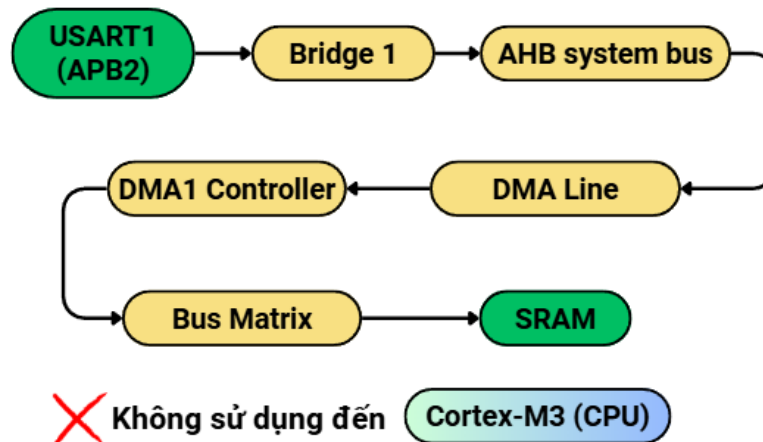
1.1.3. Sơ đồ kiến trúc Ma trận Bus (Bus Matrix Architecture) trong STM32F1:

- Sơ đồ kiến trúc hệ thống:



- Trong đó bao gồm các thành phần chính sau:
 - **Cortex-M3:** Nhân vi xử lý trung tâm (CPU)
 - **DMA1 / DMA2:** Các bộ điều khiển DMA.
Lưu ý: Chip STM32F103C8T6 của bạn thuộc dòng Medium-density, nên nó chỉ có DMA1 (với 7 kênh). Khối DMA2 trong hình chỉ có ở các dòng High-density (như F103RCT6, RET6 trở lên).
 - **Bus Matrix (Ma trận Bus):** Đây là "trái tim" điều phối giao thông, cho phép CPU và DMA truy cập bộ nhớ song song nhờ cơ chế trọng tài (Arbiter).
 - **Flash:** Bộ nhớ chứa chương trình (Code) của bạn. Nó được kết nối qua giao diện FLITF (Flash Memory Interface) để CPU có thể đọc lệnh.
 - **SRAM:** Bộ nhớ RAM (Static RAM). Đây là nơi chứa biến số, Stack, Heap. Trong hình, nó là đích đến của đường mũi tên vàng.
 - **Bridge 1 & Bridge 2 (Cầu nối AHB-APB):**
 - Chuyển đổi giao thức từ bus AHB tốc độ cao sang bus ngoại vi APB.
 - **Bridge 1 (APB2):** Nối ra các ngoại vi tốc độ cao (Max 72MHz).
 - **Bridge 2 (APB1):** Nối ra các ngoại vi tốc độ thấp hơn (Max 36MHz).

- Các ngoại vi như ADC, UART, SPI,... nằm trên các đường Bus tương ứng của chúng
- **DMA Request:** Các đường mũi tên mỏng từ ngoại vi về khối DMA. Đây là dây tín hiệu "báo cáo" khi ngoại vi cần chuyển dữ liệu (Ví dụ: ADC báo "tôi chuyển đổi xong rồi").
- Đường đi của dữ liệu do DMA điều khiển:
Ví dụ: Ở đây ta xét dữ liệu ở UART gửi về SRAM thông qua UART.



➡ Giải phóng CPU cho thuật toán điều khiển

- **Bước 1: Nguồn dữ liệu & Khởi tạo Yêu cầu (Source Peripheral & Request)**
 - Dữ liệu nhận được bắt đầu tại **USART1** (khối màu vàng dưới cùng).
 - USART1 nằm trên **Bus APB2** (Advanced Peripheral Bus 2). Đây là bus ngoại vi tốc độ cao (High-speed), hoạt động ở tần số tối đa 72MHz (trên STM32F1).
 - Nó gửi tín hiệu DMA Request (đường mũi tên mỏng) đến bộ điều khiển DMA1.
 - Bộ DMA1 (đóng vai trò là Master) tiếp nhận yêu cầu và chuẩn bị thực hiện lệnh chuyển hàng.
- **Bước 2: Master xin đường & Phân xử (Arbitration at Bus Matrix)**
 - DMA1 gửi tín hiệu xin truy cập vào **Bus Matrix** (Ma trận Bus)
 - Tại đây, **Bộ phân xử (Arbiter)** sẽ quyết định quyền đi tiếp dựa trên thuật toán **Round-Robin** (Xoay vòng).
 - Nếu CPU đang rảnh: DMA được cấp quyền ngay.

- Nếu CPU cũng đang truy cập bộ nhớ: Bộ phân xử sẽ chia sẻ băng thông, đảm bảo CPU luôn có ít nhất 50% thời gian truy cập để không bị treo hệ thống.
- **Bước 3: Qua cầu nối (AHB-APB Bridge) & Vận chuyển**
 - Sau khi được cấp quyền, dữ liệu từ USART1 đi qua **Bridge 1** (Cầu nối 1).
 - **Chức năng:** Bộ cầu nối này chuyển đổi giao thức từ bus chậm (APB2) sang giao thức của bus hệ thống tốc độ cao (AHB System Bus). Nó đóng vai trò đồng bộ hóa miền xung nhịp (Clock Domain Crossing) giữa ngoại vi và hệ thống.
 - Dữ liệu đi vào **Bus Matrix** thông qua cổng AHB system bus.
 - Tại đây, DMA1 "lái" dòng dữ liệu đi xuyên qua ma trận theo đường dẫn màu vàng (The Yellow Path).
- **Bước 4: Đích đến (Destination Memory)**
 - Từ Ma trận Bus, dữ liệu được ghi trực tiếp vào **SRAM** (khối màu vàng trên cùng bên phải).
 - Nếu bạn đã cấu hình bit MINC=1 (Memory Increment), địa chỉ con trỏ tại SRAM sẽ tự động tăng lên để chuẩn bị cho byte tiếp theo.
 - Quá trình này diễn ra song song và độc lập với việc nạp lệnh của CPU.
- **Bước 5: Bắt tay & Kết thúc (Handshake & Completion) – Cơ chế phản hồi**
 - Ngay khi DMA truy cập thành công vào thanh ghi dữ liệu của USART1, nó gửi ngược lại tín hiệu **Acknowledge** (Xác nhận) cho USART1.
 - USART1 nhận được Acknowledge sẽ **tắt tín hiệu DMA Request** (Release request) và xóa các cờ ngắt liên quan, sẵn sàng cho lần nhận dữ liệu tiếp theo.

1.2. Cấu Trúc và Hoạt Động Của DMA Controller:

1.2.1. Bộ điều khiển DMA1 (Trên STM32F103C8T6):

- Vi điều khiển STM32F103C8T6 thuộc dòng *Medium-density*, do đó nó được trang bị **1 bộ DMA (DMA1)** với **7 kênh (Channels)** độc lập. (*Lưu ý: Các dòng High-density mới có thêm DMA2 với 5 kênh nữa*).
- Trên dòng F1, **mỗi kênh DMA được gán cứng (Hard-wired) cho một số ngoại vi nhất định**. Không thể thay đổi luồng này bằng phần mềm
- **Ví dụ:**
 - Channel 1: ADC1, TIM2_CH3, TIM4_CH1.
 - Channel 4: USART1_TX, I2C2_TX, TIM1_CH4.

- Ta có sơ đồ DMA Request Mapping trong Datasheet của STM32F10x như sau:

Summary of DMA1 requests for each channel							
Peripherals	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Channel 6	Channel 7
ADC1	ADC1	-	-	-	-	-	-
SPI/I ² S	-	SPI1_RX	SPI1_TX	SPI2/I2S2_RX	SPI2/I2S2_TX	-	-
USART	-	USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX
I ² C	-	-	-	I2C2_TX	I2C2_RX	I2C1_TX	I2C1_RX
TIM1	-	TIM1_CH1	-	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	-
TIM2	TIM2_CH3	TIM2_UP	-	-	TIM2_CH1	-	TIM2_CH2 TIM2_CH4
TIM3	-	TIM3_CH3	TIM3_CH4 TIM3_UP	-	-	TIM3_CH1 TIM3_TRIG	-
TIM4	TIM4_CH1	-	-	TIM4_CH2	TIM4_CH3	-	TIM4_UP

1.2.2. Các thanh ghi chính (Core Register):

- Mỗi kênh DMA (x = 1..7) được điều khiển bởi 4 thanh ghi chính:
 - **DMA_CCRx (Configuration Register):** Thanh ghi quan trọng nhất. Cấu hình độ ưu tiên, hướng truyền, chế độ (Circular/Normal), kích thước dữ liệu (8/16/32 bit), và cho phép ngắt.
 - **DMA_CNDTRx (Number of Data Register):** Chứa số lượng dữ liệu cần truyền (từ 0 đến 65535). Giá trị này giảm dần sau mỗi lần truyền.
 - **DMA_CPARx (Peripheral Address Register):** Chứa địa chỉ cơ sở của thanh ghi dữ liệu ngoại vi (Ví dụ: &ADC1->DR).
 - **DMA_CMARx (Memory Address Register):** Chứa địa chỉ cơ sở của vùng nhớ đệm trong RAM (Ví dụ: &myBuffer[0]).

1.2.3. Trọng số ưu tiên (Arbitration)

- Bộ điều khiển quản lý các yêu cầu kênh dựa trên mức độ ưu tiên và khởi chạy các chuỗi truy cập thiết bị ngoại vi/bộ nhớ. Khi nhiều kênh DMA yêu cầu truyền cùng lúc, sự tranh chấp được giải quyết qua 2 bước:
 - **Software Priority (PL[1:0]):** Cài đặt trong thanh ghi CCR với 4 mức: *Very High, High, Medium, Low*.
 - **Hardware Priority:** Nếu mức phần mềm bằng nhau, kênh có chỉ số thấp hơn được ưu tiên (Kênh 1 > Kênh 2 > ... > Kênh 7).

1.2.4. Giao tiếp dữ liệu DMA:

- Mỗi lần truyền dữ liệu DMA gồm có các thao tác sau:
 - Việc **tải dữ liệu từ thanh ghi dữ liệu ngoại vi hoặc một vị trí trong bộ nhớ** được truy cập thông qua **thanh ghi địa chỉ ngoại vi/bộ nhớ hiện tại** bên trong. Địa chỉ bắt đầu được sử dụng cho lần truyền đầu tiên là địa chỉ ngoại

vi/bộ nhớ cơ sở được **lập trình** trong thanh ghi **DMA_CPARx** hoặc **DMA_CMARx**.

- Dữ liệu được lưu trữ vào thanh ghi dữ liệu ngoại vi hoặc một vị trí trong bộ nhớ được định địa chỉ thông qua thanh ghi địa chỉ ngoại vi/bộ nhớ hiện tại bên trong. Địa chỉ bắt đầu được sử dụng cho lần truyền đầu tiên là địa chỉ ngoại vi/bộ nhớ cơ sở được lập trình trong **thanh ghi DMA_CPARx** hoặc **DMA_CMARx**.
- Việc **giảm** giá trị của **thanh ghi DMA_CNDTRx**, chứa số lượng giao dịch vẫn chưa được thực hiện, là quá trình thực hiện tiếp theo.

1.3. Các chế độ hoạt động:

1.3.1. Hướng truyền dữ liệu (Data Direction)

- Peripheral to Memory (P2M)

- **Cấu hình:** Bit DIR = 0 (đọc từ ngoại vi).
- **Cơ chế:** DMA chờ tín hiệu yêu cầu (Request) từ ngoại vi (ví dụ: ADC chuyển đổi xong). Khi có tín hiệu, DMA đọc dữ liệu từ thanh ghi dữ liệu của ngoại vi (&ADC->DR) và ghi vào địa chỉ RAM được chỉ định.
- **Ví dụ:** Đọc giá trị từ cảm biến nhiệt độ qua ADC và lưu vào biến uint16_t temp_val trong RAM.

- Memory to Peripheral (M2P)

- **Cấu hình:** Bit DIR = 1 (đọc từ bộ nhớ).
- **Cơ chế:** DMA chờ tín hiệu từ ngoại vi báo "đang rảnh" (ví dụ: thanh ghi đệm phát của UART TXE đang trống). Khi rảnh, DMA lấy dữ liệu từ mảng trong RAM và ghi vào thanh ghi ngoại vi (&USART->DR).
- **Ví dụ:** Gửi chuỗi ký tự "Hello World" ra máy tính qua giao tiếp UART.

- Memory to Memory (M2M)

- **Cấu hình:** Bit MEM2MEM = 1.
- **Cơ chế:** Khác với 2 hướng trên, chế độ này **không phụ thuộc vào ngoại vi**. Ngay khi bit EN (Enable) được bật, DMA sẽ chạy hết tốc lực để copy dữ liệu từ địa chỉ nguồn sang địa chỉ đích trong RAM (hoặc Flash sang RAM).
- **Ví dụ:** Copy dữ liệu từ bộ đệm nhận (Rx Buffer) sang bộ đệm xử lý (Processing Buffer) để giải phóng Rx Buffer cho dữ liệu mới.

1.3.2. Chế Độ Hoạt Động (Mode Selection)

- Normal Mode (Chế độ thường - CIRC = 0):

- Khi DMA được kích hoạt, thanh ghi đếm CNDTR (Number of Data to Transfer) sẽ giảm dần từ giá trị cài đặt về 0.
- Khi **CNDTR = 0**, quá trình truyền dừng lại hoàn toàn.

- Muốn truyền lại, CPU phải: Tắt DMA -> Ghi lại giá trị vào **CNDTR** -> Bật lại DMA.
- **Ứng dụng:** Gửi gói tin UART một lần, copy dữ liệu bộ nhớ.
- **Circular Mode (Chế độ vòng tròn - CIRC = 1):**
 - Khi CNDTR giảm về 0, DMA Controller sẽ **tự động nạp lại** giá trị ban đầu cho CNDTR và đưa con trỏ bộ nhớ về vị trí đầu tiên.
 - Quá trình này lặp đi lặp lại vô tận mà không cần CPU can thiệp.
 - **Ứng dụng:** Đọc ADC liên tục (chức năng như máy hiện sóng), tạo bộ đệm vòng (Ring Buffer) cho UART Rx để không bị mất dữ liệu.

1.3.3. Quản Lý Con Trỏ (Increment Mode)

- DMA cần biết sau khi chuyển 1 dữ liệu, nó nên giữ nguyên địa chỉ hay nhảy sang địa chỉ tiếp theo.
- **Peripheral Increment (PINC):** Tăng địa chỉ ngoại vi.
 - **Thường là DISABLED:** Vì ngoại vi (như ADC, UART) chỉ có **một** thanh ghi dữ liệu cố định (ví dụ 0x4001244C). Nếu tăng địa chỉ, DMA sẽ đọc sai sang thanh ghi khác.
 - **Trường hợp ENABLED:** Rất hiếm, chỉ dùng khi ngoại vi hỗ trợ vùng nhớ FIFO lớn.
- **Memory Increment (MINC):** Tăng địa chỉ bộ nhớ.
 - **Thường là ENABLED:** Vì dữ liệu thường được lưu vào một Mảng (Array) trong RAM. Sau khi ghi vào buffer[0], DMA cần tự tăng địa chỉ để ghi tiếp vào buffer[1].
 - **Trường hợp DISABLED:** Khi bạn muốn ghi đè liên tục vào một biến đơn duy nhất.

1.4. Tương Tác Giữa DMA và CPU:

1.4.1. Các Cờ Ngắt (Interrupt Flags):

Trong thanh ghi **DMA_ISR** (Interrupt Status Register), có 3 cờ quan trọng tương ứng với mỗi kênh:

- **TC (Transfer Complete):**
 - **Ý nghĩa:** Bật lên khi DMA đã chuyển xong toàn bộ số lượng byte được yêu cầu (khi CNDTR giảm về 0).
 - **Ứng dụng:** Báo cho CPU biết "Dữ liệu đã sẵn sàng, hãy xử lý đi" hoặc "Đã gửi xong dữ liệu".
- **HT (Half Transfer):**
 - **Ý nghĩa:** Bật lên khi DMA đã chuyển được đúng **một nửa** số lượng yêu cầu (khi CNDTR giảm còn 1/2 giá trị ban đầu).

- **Ứng dụng:** Đây là "vũ khí bí mật" của DMA. Nó cho phép CPU bắt đầu xử lý nửa đầu dữ liệu trong khi DMA vẫn đang lặng lẽ điền nốt nửa sau.
- **TE (Transfer Error):**
 - **Ý nghĩa:** Báo lỗi khi DMA cố gắng truy cập vào vùng nhớ không được phép hoặc bị lỗi Bus.

1.4.2. Kỹ Thuật Buffer Kép (Simulated Double Buffering)

Trên STM32F1, chúng ta thường mô phỏng kỹ thuật này bằng một mảng lớn và sử dụng cờ HT/TC.

- **Thiết lập:** Tạo một mảng lớn có kích thước 2N (Ví dụ: 100 phần tử).
- **Hoạt động:**
 - DMA bắt đầu điền dữ liệu từ phần tử 0 đến 49.
 - Khi điền đến phần tử 49 -> Cờ HT bật -> Ngắt xảy ra. Trong ngắt này, CPU xử lý dữ liệu từ 0-49. DMA tiếp tục điền từ 50-99.
 - Khi điền đến phần tử 99 -> Cờ TC bật -> Ngắt xảy ra. Trong ngắt này, CPU xử lý dữ liệu từ 50-99. DMA (Circular mode) quay lại điền từ 0.
- **Hiệu quả:** CPU và DMA làm việc song song ("gối đầu" nhau), dữ liệu không bao giờ bị gián đoạn.

1.5. Cấu hình DMA trên STM32:

- **Bước 1: Thiết lập địa chỉ Ngoại vi (Peripheral Address)**
 - Ghi địa chỉ cơ sở của thanh ghi dữ liệu ngoại vi vào thanh ghi **DMA_CPARx**.
 - **Ý nghĩa:** Đây sẽ là địa chỉ nguồn (Source) hoặc đích (Destination) của dữ liệu tùy thuộc vào hướng truyền, và địa chỉ này sẽ được truy cập mỗi khi có yêu cầu (Request) từ ngoại vi.
- **Bước 2: Thiết lập địa chỉ Bộ nhớ (Memory Address)**
 - Ghi địa chỉ cơ sở của vùng nhớ (biến đơn hoặc mảng đệm) vào thanh ghi **DMA_CMARx**.
 - **Ý nghĩa:** Dữ liệu sẽ được đọc từ hoặc ghi vào địa chỉ này sau mỗi sự kiện ngoại vi.
- **Bước 3: Cài đặt số lượng dữ liệu (Data Quantity)**
 - Ghi tổng số lượng dữ liệu cần truyền vào thanh ghi **DMA_CNDTRx**.
 - **Cơ chế:** Giá trị của thanh ghi này sẽ tự động giảm đi 1 sau mỗi lần truyền dữ liệu thành công.
- **Bước 4: Cấu hình độ ưu tiên (Channel Priority)**
 - Thiết lập mức độ ưu tiên cho kênh DMA thông qua các bit **PL[1:0]** trong thanh ghi cấu hình **DMA_CCRx**.
 - **Các mức:** Thấp (Low), Trung bình (Medium), Cao (High), và Rất cao (Very High).

- **Bước 5: Cấu hình các tham số hoạt động (Operation Parameters)** Tiến hành cài đặt các thông số chi tiết trong thanh ghi **DMA_CCRx**:
 - **Hướng truyền dữ liệu (Data Direction)**: Từ Ngoại vi vào Bộ nhớ hay ngược lại.
 - **Chế độ vòng lặp (Circular Mode)**: Cho phép lặp lại chu trình hay dừng khi đếm hết.
 - **Chế độ tăng địa chỉ (Increment Mode)**: Có tự động tăng địa chỉ Ngoại vi (PINC) và Bộ nhớ (MINC) sau mỗi lần truyền hay không.
 - **Kích thước dữ liệu (Data Width)**: Định dạng độ rộng dữ liệu (8-bit, 16-bit, 32-bit) cho cả phía Ngoại vi (PSIZE) và Bộ nhớ (MSIZE).
 - **Cấu hình Ngắt (Interrupts)**: Cho phép ngắt khi truyền được một nửa (HTIE), hoàn thành (TCIE) hoặc khi có lỗi (TEIE).
- **Bước 6: Kích hoạt kênh (Enable Channel)**
 - Thiết lập bit **EN** trong thanh ghi **DMA_CCRx** lên mức 1.
 - *Lưu ý*: Ngay sau khi bit này được set, kênh DMA sẽ bắt đầu sẵn sàng phục vụ các yêu cầu truyền dữ liệu.
- **Lưu ý**: Một khi bit EN trong thanh ghi **DMA_CCRx** đã được bật (Enabled), các cấu hình tại bước 1, 2, 3 và 5 sẽ bị khóa và không thể thay đổi cho đến khi tắt kênh DMA (Disable)."

1.6. Các thanh ghi DMA cần sử dụng:

1.6.1. Thanh ghi cho phép clock ngoại vi **RCC_AHBENR**:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved					SDIO EN	Res.	FSMC EN	Res.	CRCE N	Res.	FLITF EN	Res.	SRAM EN	DMA2 EN	DMA1 EN
					rw		rw		rw		rw		rw	rw	rw

- Cho phép clock DMA1: **RCC->AHBENR [0] = 1**

1.6.2. Thanh ghi địa chỉ ngoại vi - **DMA channel x peripheral address register (DMA_CPARx)**:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PA																															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **PA[31:0]**: Peripheral address

- **Tên**: DMA Channel x **Peripheral Address Register**.
- **Chức năng**: Chứa **địa chỉ của thanh ghi dữ liệu bên phía Ngoại vi** (ADC, UART, SPI, I2C...).

- **Ví dụ Code:** Nếu muốn dùng DMA để đọc dữ liệu từ bộ chuyển đổi **ADC1**.
 - Ngoại vi: ADC1.
 - Thanh ghi chứa kết quả chuyển đổi: DR (Data Register).
 - **Giá trị nạp vào CPAR:**

// Dấu & nghĩa là "Lấy địa chỉ"

DMA1_Channel1->CPAR = (uint32_t)&(ADC1->DR);

// Thực tế nó sẽ là một con số kiểu 0x4001244C (Địa chỉ cứng trong chip)

1.6.3. Thanh ghi địa chỉ bộ nhớ - DMA channel x memory address register (DMA_CMARx):

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MA																															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:0 **MA[31:0]**: Memory address

- **Tên:** DMA Channel x **Memory** Address Register.
- **Chức năng:** Chứa **địa chỉ của biến hoặc mảng nằm trong Bộ nhớ RAM** (SRAM) mà bạn đã khai báo trong code.
- **Ví dụ Code:** Đầu tiên, khai báo một mảng để chứa 10 giá trị đo từ cảm biến:


```
uint16_t adc_buffer[10]; // Mảng này nằm trong RAM
```

 - Giá trị nạp vào CMAR:


```
// Trong C, tên mảng chính là địa chỉ phần tử đầu tiên
```

 DMA1_Channel1->CMAR = (uint32_t)adc_buffer;

1.6.4. Thanh ghi số lượng của dữ liệu - DMA channel x number of data register (DMA_CNDTRx):

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

- **Chức năng:** Chứa số lượng dữ liệu (Number of Data) cần truyền.
- **Độ rộng:** Chỉ sử dụng 16 bit thấp (NDT[15:0]), tức là giá trị tối đa bạn có thể cài đặt là **65535** đơn vị dữ liệu
- **Cơ chế hoạt động:**
 - Thanh ghi này giống như một cái **đồng hồ đếm ngược (Down Counter)**
 - **Cài đặt ban đầu:** Trước khi bật DMA, bạn nạp số lượng cần truyền vào đây.
 - Ví dụ: Muốn đọc 10 mẫu ADC => Ghi số 10 vào CNDTRx.

- **Trong khi chạy:**
 - Mỗi khi DMA chuyển thành công 1 đơn vị dữ liệu (Byte/Half-word/Word tùy theo PSIZE/MSIZE), giá trị trong thanh ghi này sẽ **tự động giảm đi 1**.
 - Ví dụ: Chuyển xong mẫu 1 => CNDTR còn 9.
- **Khi về 0 (Zero):**
 - Quá trình truyền DMA dừng lại.
 - Cờ ngắt TCIF (Transfer Complete) được bật lên để báo cho CPU biết.

1.6.5. Thanh ghi cấu hình cho DMA - DMA channel x configuration register (DMA_CCRx):

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	MEM2 MEM	PL[1:0]		MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR	TEIE	HTIE	TCIE	EN
	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

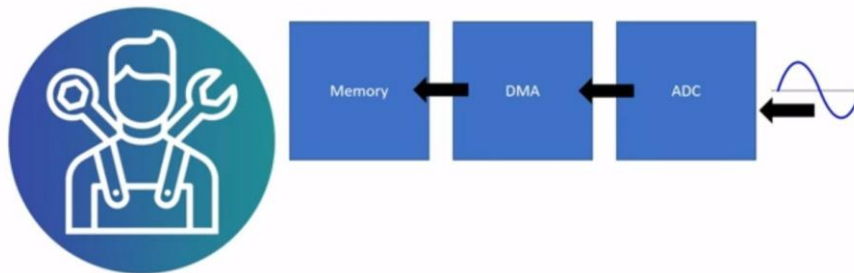
- Chức năng: Thanh ghi cấu hình trong DMA, cho phép cấu hình các tham số như hướng dữ liệu, chế độ, ..
- Các bit và chức năng của các bit trong thanh ghi:
 - **Bit EN (DMA_CCRx[0]):** Bit điều khiển kích hoạt DMA
 - 0: Kênh tắt.
 - 1: Kênh bật & bắt đầu làm việc.
 - **Nguyên tắc vàng:** Chỉ được phép sửa đổi các bit cấu hình bên trên (MSIZE, DIR, CIRC...) khi **EN = 0**. Nếu EN đang là 1 mà lại cố ghi vào thanh ghi này, chip sẽ lờ đi thao tác đó.
 - **Bit DIR (DMA_CCRx[4]):** Cấu hình hướng truyền dữ liệu:
 - 0: **Read from Peripheral** (Ngoại vi -> Bộ nhớ). Ví dụ: Đọc ADC, Nhận UART Rx.
 - 1: **Read from Memory** (Bộ nhớ -> Ngoại vi). Ví dụ: Gửi UART Tx, Điều khiển tốc độ qua Timer CCR.
 - **Bit CIRC (DMA_CCRx[5]):** Cấu hình chế độ vòng tròn
 - 0 (Normal): Chạy hết số lượng mẫu quy định rồi dừng, tắt kênh. Dừng khi truyền gói tin sạch.
 - 1 (Circular): Chạy hết rồi tự động quay lại từ đầu, chạy mãi mãi. **Bắt buộc dừng** khi quét cảm biến liên tục hoặc đẩy dữ liệu ra PWM/DAC liên tục.
 - **Bit PINC (DMA_CCRx[6]):** Chế độ tăng địa chỉ cho con trỏ ngoại vi
 - 0: Tắt.
 - 1: Bật.

- *Thực tế*: Bit này **thường xuyên là 0**. Vì ngoại vi (như ADC, UART) chỉ có 1 thanh ghi dữ liệu (DR) nằm cố định tại một địa chỉ. Nếu cho nó tự tăng, DMA sẽ đọc sai sang các thanh ghi điều khiển bên cạnh.
- **Bit MINC (DMA_CCRx[7])**: Chế độ tăng địa chỉ cho con trỏ bộ nhớ
 - 0: Tắt (Ghi mãi vào 1 chỗ).
 - 1: Bật (Địa chỉ tự tăng).
 - *Thực tế*: Để lưu dữ liệu, bit này **luôn luôn là 1**. Nếu để 0, mọi giá trị cảm biến sẽ ghi đè lên nhau tại phần tử đầu tiên buffer[0].
- **Các bit MSIZE (DMA_CCRx[11:10])**: Độ rộng dữ liệu tại địa chỉ vùng nhớ
 - 00: 8-bit (Byte) -> Tương ứng biến uint8_t.
 - 01: 16-bit (Half-word) -> Tương ứng biến uint16_t.
 - 10: 32-bit (Word) -> Tương ứng biến uint32_t.
 - *Lưu ý*: Nếu khai báo mảng lưu trữ kiểu gì thì chọn bit này tương ứng.
- **Các bit PSIZE (DMA_CCRx[8:9])**: Độ rộng dữ liệu tại địa chỉ ngoại vi
 - Giá trị cấu hình tương tự (00, 01, 10).
 - *Ví dụ*: ADC1 của STM32F1 có độ phân giải 12-bit, dữ liệu lưu trong thanh ghi 16-bit, nên bạn **phải chọn 01 (16-bit)**. Nếu chọn sai (ví dụ 8-bit), sẽ bị mất dữ liệu.

1.7. Cấu hình ADC DMA:

ADC DMA

ADC with DMA



1.7.1. Lý do nên sử dụng ADM DMA:

- Trong các hệ thống nhúng, việc thu thập dữ liệu từ cảm biến (Sensor) thường gặp các thách thức lớn:
 - **Đa kênh (Multi-channel)**: Vi điều khiển cần đọc cùng lúc nhiều cảm biến (Nhiệt độ, độ ẩm, dòng điện, điện áp...).
 - **Tần số lấy mẫu cao (High Sampling Rate)**: Việc đọc liên tục bằng phương pháp Polling hoặc Ngắt (Interrupt) sẽ chiếm dụng hoàn toàn CPU, khiến hệ

thống không thể xử lý các tác vụ khác (như tính toán PID, hiển thị LCD, truyền thông).

⇒ Sử dụng cơ chế ADC kết hợp DMA. Khi đó, mỗi khi ADC chuyển đổi xong một giá trị, nó sẽ tự động gửi một tín hiệu yêu cầu (DMA Request) để DMA mang giá trị đó cất vào RAM ngay lập tức.

1.7.2. Quá trình truyền dữ liệu:

- **Bước 1 - Chuyển đổi hoàn tất (EOC):** ADC hoàn thành việc chuyển đổi tín hiệu analog sang số cho một kênh (ví dụ: Kênh 0). Dữ liệu được lưu vào thanh ghi ADC_DR.
- **Bước 2 - Phát sinh yêu cầu (DMA Request):** Ngay khi có dữ liệu mới trong ADC_DR, khối ADC phát một tín hiệu yêu cầu đến bộ điều khiển DMA.
- **Bước 3 - Vận chuyển (Transfer):** DMA nhận yêu cầu, chiếm quyền Bus, đọc giá trị từ ADC_DR và ghi vào địa chỉ bộ nhớ hiện tại (ví dụ: buffer[0]).
- **Bước 4 - Tăng địa chỉ:** Nếu được cấu hình MINC (Memory Increment), con trỏ địa chỉ bộ nhớ sẽ tăng lên (trở sang buffer[1]).
- **Bước 5 - Lặp lại:** ADC tiếp tục chuyển đổi kênh tiếp theo (Kênh 1) và quy trình lặp lại.

1.7.3. Các chế độ ADC khi dùng DMA:

- **Scan Mode (Chế độ Quét)**
 - **Mô tả:** Cho phép ADC chuyển đổi lần lượt một nhóm các kênh (Ví dụ: Ch0 -> Ch1 -> Ch2) chỉ với một lệnh kích hoạt.
 - **Kết hợp DMA:** Đây là chế độ bắt buộc khi đọc nhiều kênh. DMA sẽ lần lượt xếp các giá trị Ch0, Ch1, Ch2 vào mảng buffer[0], buffer[1], buffer[2] tương ứng.
- **Continuous Conversion Mode (Chế độ Chuyển đổi Liên tục)**
 - **Mô tả:** Ngay sau khi chuyển đổi xong kênh cuối cùng của nhóm, ADC tự động quay lại kênh đầu tiên và bắt đầu vòng chuyển đổi mới ngay lập tức.
 - **Kết hợp DMA Circular Mode:** Tạo ra một hệ thống giám sát thời gian thực (Real-time monitoring). Mảng dữ liệu trong RAM luôn được cập nhật giá trị mới nhất từ cảm biến mà không cần bất kỳ dòng code nào từ CPU.

1.7.4. Cấu hình DMA trong thanh ghi của ADC:

Cấu hình DMA trong thanh ghi ADC control register 2 (ADC_CR2):

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								TSVRE FE	SWSTA RT	JSWST ART	EXTTR IG	EXTSEL[2:0]			Res.
								rw	rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JEXTT RIG	JEXTSEL[2:0]			ALIGN	Reserved		DMA	Reserved				RST CAL	CAL	CONT	ADON
rw	rw	rw	rw	rw	Res.		rw					rw	rw	rw	rw

Bật chế độ ADC DMA

Tại đây, ta chỉ cần bật bit DMA trong ADC_CR2, lúc này DMA sẽ tự động copy dữ liệu từ thanh ghi ngoại vi ADC vào đích đến cần lưu trong SRAM